Synchronous Message Delivery in Actor Systems with Receiver Transparency and Cyclic-Invocation Detection

The `fast_send` Mechanism

Vincent Maciejewski
M2 Technologies
mayeski@gmail.com

DRAFT — September 2026

Abstract

The actor model isolates concurrent entities behind asynchronous message passing: a sender enqueues a message on the receiver's mailbox and continues, and the receiver processes it later on its own thread. For actors that reside in the same process on the same machine, this discipline imposes overhead that the isolation guarantee does not require — heap allocation of every message, two queue operations, a scheduler dispatch, and a context switch per delivery, with a second round trip for any reply. We describe `fast_send`, a synchronous delivery mechanism in which the sending thread acquires exclusive access to the receiving actor and executes the receiver's message handler directly, on the sender's own thread, returning the reply as a value. The defining property is *receiver transparency*: the handler is written identically for both delivery modes and has no programmatic means to determine which was used, which thread executes it, or whether the sender is blocked awaiting a reply. A unified reply operation routes responses correctly in both modes, and the reply value is heap-allocated in both modes so that the handler cannot infer the mode from allocation behavior. Synchronous delivery across a chain of actors runs entirely on the first actor's thread when the chain's locks are uncontended, reducing scheduler context switches from $O(N)$ on the asynchronous path to $O(1)$. Because synchronous invocation can form cycles that would deadlock a non-reentrant lock or overflow the stack under a reentrant one, the mechanism carries a thread-local call-chain membership test that detects a cyclic invocation *before* the target lock is acquired, and either raises a cyclic invocation exception or, in a best-effort variant, falls back to asynchronous delivery. A refined variant keys the call chain on (actor, message-class) pairs to admit legitimate callback patterns while still catching unbounded recursion. Five delivery operations — asynchronous, blocking synchronous, spin-wait synchronous, non-blocking synchronous, and best-effort void — expose the full latency/robustness spectrum behind one transparent handler interface. We give the mechanism, the detection algorithm, and an analytical account of the per-message operations eliminated, corroborated by microbenchmarks of the C++ implementation in which the synchronous path completes a round trip in tens of nanoseconds — roughly 60× below the ungrouped asynchronous path — and a memory pool cuts the latency tail by two orders of magnitude; a saturated-throughput and contention evaluation is left to future work.

Keywords: actor model, message passing, concurrency, synchronous delivery, deadlock detection, receiver transparency, low-latency systems.

1 Introduction

In the actor model [1, 2], a program is a collection of isolated actors, each with private state, a message handler, and a mailbox. Actors communicate only by asynchronous message passing: a sender places a message on the receiver's mailbox and proceeds without waiting, and the receiver dequeues and processes messages one at a time on its own thread. This discipline is what gives the model its guarantees — isolation, sequential per-actor processing, and location transparency — and it is essential when actors are distributed across machines.

When the communicating actors reside in the same process on the same machine, however, the asynchronous path carries overhead that the isolation guarantee does not require. A single request-and-reply between two co-located actors incurs, on conventional implementations:

- **Heap allocation.** Because the message must outlive the sender's current continuation and persist until processed at an indeterminate later time, it must be heap-allocated, with the associated allocation cost, fragmentation, and garbage-collection pressure.
- **Queue operations.** Each message is enqueued by the sender and dequeued by the receiver, each requiring synchronization on the mailbox.
- **Scheduling and context switches.** The receiver must be scheduled onto a thread to run, incurring a context switch; a context switch typically costs on the order of 10^3–10^4 processor cycles and pollutes cache and translation lookaside buffer (TLB) state.
- **Round-trip latency.** A reply repeats the entire path in reverse, so a request-and-reply crosses the mailbox twice and the scheduler twice.

For a co-located pair, all of this is avoidable in principle: the sender's own thread could run the receiver's handler directly and take the reply as a return value. The difficulty is doing so without breaking the abstraction the actor model exists to provide. A direct handler invocation — calling the receiver's handler as an ordinary method — discards the receiver's exclusive-execution guarantee and exposes the receiver as a passive object. The design problem is to obtain the performance of a direct, same-thread call while preserving, from the receiver's point of view, the semantics of message passing.

This paper describes `fast_send`, a synchronous delivery mechanism that resolves that tension. The sending thread acquires exclusive access to the receiving actor and executes the receiver's handler directly, on the sender's thread, returning the reply as a value. The receiver's handler is identical for synchronous and asynchronous delivery and cannot determine which was used — the property we call *receiver transparency*. Because synchronous invocations can form cycles, and a cycle deadlocks a non-reentrant lock or overflows the stack under a reentrant one, the mechanism carries a lightweight cyclic-invocation detector that fires before any lock is taken.

Contributions.

1. A synchronous, same-thread actor-delivery mechanism (`fast_send`) that eliminates the heap allocation, queue operations, scheduling, and context switches of the asynchronous path for co-located actors, and returns the reply as a value (Section 6).
2. A precise statement and enforcement of *receiver transparency*: the handler executes identical code in both modes and cannot determine the delivery mode, the executing thread, or whether the sender is blocked (Section 7).

3. A thread-local call-chain membership test that detects cyclic synchronous invocation before lock acquisition, together with a refined (actor, message-class) variant that admits callback patterns while still catching unbounded recursion (Section 8).

4. A family of five delivery operations spanning asynchronous through spin-wait synchronous to best-effort-with-async-fallback, unified behind one transparent handler interface (Section 9).

5. An analytical account of the per-message work eliminated, together with microbenchmarks of the C++ implementation that measure each eliminated operation and the pool's tail benefit; a saturated-throughput evaluation is left to future work (Section 11).

2 Origins and history of the actor model

Invention (1973). The actor model was introduced by Carl Hewitt, Peter Bishop, and Richard Steiger in "A Universal Modular ACTOR Formalism for Artificial Intelligence," presented at the Third International Joint Conference on Artificial Intelligence (IJCAI) at Stanford in August 1973 [1]. Its origins were in artificial-intelligence research at MIT: the formalism grew out of Hewitt's earlier work on the PLANNER language and was proposed as a single, uniform notion — the *actor*, an entity that communicates only by sending messages to other actors — rich enough to serve as a foundation for AI programming. Hewitt drew on the object and message-sending ideas of Simula 67 and Smalltalk, on Lisp, and on capability systems, Petri nets, and packet-switched networks; in later retrospectives he described the model as inspired by the laws of physics rather than by any prior computational abstraction [22].

Formal foundations (1975–1986). The 1973 paper was informal, and a decade of work followed to give the model a rigorous semantics. Irene Greif's 1975 MIT doctoral thesis supplied the first operational semantics for actor computations [18]; Hewitt and Baker stated "laws" characterizing the partial ordering of events in a distributed actor computation [19]; and William Clinger's 1981 thesis gave a denotational semantics based on power domains, addressing the unbounded nondeterminism that distinguishes actors from other concurrency formalisms [20]. The foundation most cited today is Gul Agha's doctoral work, published as *Actors: A Model of Concurrent Computation in Distributed Systems* [2], which recast the model with a transition-system operational semantics built on the primitives *create*, *send*, and *become*, and moved the emphasis from artificial intelligence to a general theory of open, concurrent, distributed computation.

From theory to practice. The most influential early *deployed* system with actor-like structure — isolated processes with private state that communicate only by asynchronous message passing — was Erlang, developed at Ericsson from 1986 by Joe Armstrong, Robert Virding, and Mike Williams for fault-tolerant telephony [5]. The historical relationship to Hewitt's model is often overstated: Erlang's designers have said they were unaware of the actor model while building the language, and Armstrong later characterized Erlang's concurrency as closer in spirit to communicating sequential processes than to the actor model [21]. Erlang is therefore best understood as a parallel, largely independent arrival at similar principles rather than a direct descendant, even though it is now routinely described as an actor language. Hewitt himself continued to revise the model's definition well after 1973, emphasizing unbounded nondeterminism, locality, and guaranteed delivery in a much later retrospective [22]. The subsequent two decades produced a broad ecosystem of practical actor runtimes — on the JVM, the .NET platform, C++, Rust, Swift, and elsewhere — which we survey next.

3 A survey of actor-model implementations

This section surveys the major actor runtimes along the three axes most relevant to `fast_send`: how actors are mapped to operating-system threads, whether and how a synchronous or request–reply path is offered, and what is known of their performance. The consistent findings across the ecosystem — that actors are multiplexed M:N over a small worker pool, and that no mainstream system runs the *receiver's* handler on the *sender's* thread — are what locate the contribution of `fast_send`. Table 1 summarizes the comparison; the per-system detail and performance figures follow. The later related-work discussion (Section 5) revisits the systems closest to `fast_send` to sharpen the distinction; here the account is descriptive.

BEAM: Erlang and Elixir. The Erlang runtime multiplexes very large numbers of lightweight processes — each with a private heap and mailbox — onto a small set of scheduler threads, one per logical core by default, with a per-scheduler run queue and work-stealing migration [23]. Scheduling is preemptive by a reduction count: a process runs until it consumes its budget (4000 reductions by default) and is then descheduled, which bounds tail latency [23]. The synchronous path, `gen_server:call`, blocks the caller for up to a timeout and returns a value, but the request is placed in the *server* process's mailbox and handled by `handle_call/3` on the server's own scheduler; the caller merely performs a selective receive on the reply. The caller's thread never runs the callee's handler [5].

JVM: Akka. Akka does not bind actors to threads. A dispatcher binds a set of actors to a fork–join thread pool, and an actor occupies a thread only while it has messages to process [7]. A `throughput` setting caps how many messages an actor processes before the thread moves to the next actor; this mailbox batching is the principal latency/throughput lever. Akka's own microbenchmark reports on the order of 50 million messages per second on a single machine in a ping-pong configuration, a figure the authors show is dominated by that batching setting; it is a vendor microbenchmark, not an application measurement, and should be read as such [24]. The request–reply operator `ask` returns a `Future/CompletionStage` rather than a value and never blocks the caller on the handler; Akka's documentation explicitly discourages blocking inside actors because it starves the shared pool [7].

.NET: Orleans and Akka.NET. Orleans introduced the "virtual actor" (grain) abstraction, in which the runtime activates grains on demand and schedules them turn-based and single-threaded over a shared pool; every grain method is asynchronous and returns a `Task`, so a caller obtains a promise and the grain body runs on the grain's own activation context, never on the caller's thread [9]. The Orleans team reported the Halo 4 presence service sustaining roughly 130,000 requests per second across 25 servers with near-linear scaling to 125 servers [25]. A recent independent empirical comparison of Akka, Akka.NET, and Orleans on commodity cloud hardware found Akka.NET and Orleans comparable and both ahead of Akka in a non-distributed setup, with Orleans strongest in the distributed setup [27].

C++: CAF and SObjectizer. The C++ Actor Framework schedules cooperatively-run actors over a work-stealing pool of one worker per core, with a lock-free mailbox; actors that must block opt into a `detached` mode that gives them a dedicated thread [8]. Its request–reply operation, `request`, does not occupy a CAF thread while waiting: the requesting actor simply processes no other message until the response arrives, and installs a typed continuation via `.then`; the caller

Table 1: Actor-to-thread mapping and synchronous messaging across implementations. The final column — whether the sender's thread ever executes the receiver's message handler — is negative for every surveyed system and positive only for `fast_send`.

System	Runtime	Actor → thread mapping	Synchronous / request–reply	Sender runs handler?
Erlang/OTP, Elixir	BEAM VM	M:N green processes; one scheduler thread per core; per-scheduler run queues, work stealing	`gen_server:call` blocks caller, returns a value; handler runs in the server process	No
Akka	JVM	thread-pool dispatchers over a fork–join pool; actors not 1:1 with threads	`ask` (`?`) returns a `Future`; `tell` (`!`) is fire-and-forget	No
Akka.NET, Orleans	.NET	thread-pool / turn-based scheduling; grains single-threaded per activation	grain methods return `Task<T>`; always asynchronous	No
CAF	C++	cooperative work-stealing scheduler over P=#cores workers; `detached` actors get a thread	`request` returns a response handle; `.then` continuation, non-blocking	No
SObjectizer	C++	agents bound to dispatchers supplying worker threads/pools	async by default; `so5extra request_reply` blocks; runs on provider's dispatcher	No
Actix	Rust / Tokio	actors are tasks on Arbiter event-loop threads (M:N)	`send` returns a `Request` future; `do_send` fire-and-forget	No
Pony	Pony RT	work-stealing scheduler, one thread per core; per-actor heaps	none — behaviours are asynchronous by design	No
Swift actors	Swift RT	serial executor per actor over a cooperative pool	cross-actor access requires `await`; synchronous cross-actor calls prohibited by the compiler	No
Ray, Dapr, Proto.Actor	distributed	one worker/process or mailbox per actor; M:N pools	remote call returns a future/promise; turn-based dispatch	No
`fast_send` (this work)	C++/Rust	per-actor exclusive-access lock; sender's thread runs the handler	blocking call returns a value on the sender's thread	**Yes**

does not execute the callee's handler [8]. On the Savina cross-language benchmarks, a recent study found CAF only marginally ahead of Akka on average (about 1.09× at 20 threads) and behind it on nine of sixteen benchmarks [16], so the common claim that native C++ actors dominate the JVM is more nuanced than it is often stated. SObjectizer likewise binds agents to dispatchers backed by worker-thread pools and is asynchronous by default; its companion library offers a synchronous

`request_reply`, but the handler still runs on the service provider's dispatcher, not the caller's thread.

Rust: Actix. Actix runs actors as tasks on Arbiter event loops atop the Tokio runtime, multiplexing many actors per thread. `send` returns a `Request` future that resolves to the handler's result, and `do_send` is fire-and-forget; in both cases the handler runs on the target actor's Arbiter, not the sender's. (The widely-cited Actix TechEmpower web-framework results concern `actix-web`, not the actor library, and are not evidence about actor-message performance.)

Pony. Pony is the clearest case of the mainstream design point. Behaviours are asynchronous by construction — there is no synchronous send at all — so request–reply must be built by hand from callback messages. Pony runs a work-stealing scheduler with one thread per core and gives each actor its own heap, which enables a fully concurrent, per-actor garbage collector (ORCA) that needs neither read nor write barriers, relying instead on reference capabilities for data-race freedom and on causal message delivery for correctness [26]. Its performance comes from cheap actors, local GC, and never blocking.

Swift. Swift's language-level actors (SE-0306) give each actor a serial executor drawn from a cooperative thread pool. Cross-actor access must be `await`-ed and is therefore asynchronous; the compiler *prohibits* synchronous cross-actor calls outright [10]. Swift actors are reentrant: a suspended actor may interleave other work before resuming.

Distributed runtimes. Ray, Dapr, and Proto.Actor extend the actor idea across machines. A Ray actor is by default a single worker executing methods one at a time, and a call returns an object reference (a future); Dapr enforces turn-based, single-threaded access to each actor via a per-actor lock and dispatches method calls over HTTP/gRPC; Proto.Actor processes each mailbox one message at a time on a shared thread-pool dispatcher. In all three the invocation is asynchronous and the sender never runs the receiver's handler.

Two structural facts. Two observations hold across the entire survey. First, on *thread mapping*: the dominant design multiplexes many actors M:N over a small pool of worker or scheduler threads, because assigning a dedicated operating-system thread to each actor does not scale to large actor populations; the few one-thread-per-actor cases (CAF's `detached` actors, Ray's threaded actors, process-per-actor deployments) are reserved for blocking or isolation needs and are understood as exceptions. Second, on *synchronous messaging*: where a synchronous or request–reply path exists at all, it is emulated over asynchronous mailbox delivery — the caller either blocks in a receive (Erlang) or, more commonly, receives a future/promise (Akka, Orleans, CAF, Actix, Ray) — and in every case the receiver's handler runs on the receiver's own scheduled thread or context. Some systems (Pony, Swift's cross-actor calls) forbid a synchronous call altogether. No mainstream actor runtime executes the receiver's handler on the sender's thread under the receiver's exclusive-access lock. That combination — cross-thread synchronous execution under receiver transparency — is precisely the design point `fast_send` occupies, and the safety machinery of Section 8 is what it requires to be usable.

4 Why actors? The design case and its cost

A fair question precedes any discussion of making actors faster: why adopt the actor model at all, when a program can already spawn operating-system threads and share memory under mutual exclusion, or drive concurrency through an event loop such as `boost::asio`? Those mechanisms are not deficient in capability. The case for actors is not that they compute something the alternatives cannot; it is what the discipline *removes*, and the classes of error it makes unrepresentable.

What the discipline removes. Because each actor owns its state and communicates only by messages, there is no mutable state shared between actors, and therefore no memory-level data race — the defining and most pernicious failure of lock-based concurrency. Empirical study of the field bears this out: data races and deadlocks are described as "two mistakes that are hard to make with actors" [32], and a comprehensive study of 186 real-world actor bugs confirms that actor programs are markedly less susceptible to low-level data races and deadlocks than multithreaded code, even as they admit their own higher-level bug classes [33, 34]. The programmer reasons about one message at a time against a consistent private state, sequentially, and never writes a lock. The synchronization apparatus that dominates the correctness burden of hand-rolled concurrent code — choosing lock granularity, establishing a global lock order to avoid deadlock, placing memory barriers, and reasoning about every interleaving that a thread scheduler or an `asio` callback chain admits — simply does not appear in an actor program. Fault isolation follows from the same isolation: a failing actor can be restarted by a supervisor without corrupting the state of others, the foundation of the "let it crash" reliability discipline [6].

The cost. These properties are not free, and it would be dishonest to imply otherwise. The message-passing discipline that buys isolation is exactly the source of the per-message overhead this paper is concerned with: heap allocation, mailbox queue operations, scheduling, and context switches (Section 11). This is the tax paid for isolation, and on the hottest paths it has historically been the reason engineers abandon the actor model and return to shared memory and locks. The tension between the model's safety and its per-message cost is the entire subject of this paper.

The generation argument. There is a further benefit that is only now coming into view, and it may in the end be the most consequential. An actor is a small, regular unit of code: a class with private state and one handler per message type, whose entire observable contract is the set of messages it accepts and the replies it returns. This regularity makes actors unusually amenable to automatic construction by large language model (LLM) code generators, which are fluent in the paradigm: a model can scaffold an actor, write its message handlers, and — because an actor's behavior is a function from an incoming message to a reply with no hidden shared state — generate self-contained unit tests that drive messages in and assert on the replies, without the elaborate harness that testing shared-memory code requires. The contrast with the alternative is sharp and is documented. Generating *correct* lock-based concurrent code is hard even for strong models: a dedicated benchmark for concurrent code generation finds that current LLMs struggle to produce correct synchronization and repeatedly introduce concurrency errors [35]. The actor discipline eliminates precisely the hazard that both human authors and code generators handle worst — shared mutable state guarded by hand-written locks — so actor code is not only faster to produce but is likely to be more robust and closer to correct than a hand-rolled alternative built on a custom locking scheme. (We state this as a design argument; the load-bearing evidence is the demonstrated difficulty of the lock-based alternative [35], together with the measured data-race freedom of the actor approach [32, 33].)

Removing the cost makes the benefits free. Taken together, the benefits of the actor model — data-race freedom, sequential per-actor reasoning, fault isolation, and now amenability to reliable automatic generation and testing — have always been offset by a performance penalty that put the model out of reach on latency-critical paths. If that penalty can be removed for the common case of co-located actors, the calculus changes: the isolation, the absence of hand-written locking, and the generation-friendliness are retained, while the per-message cost approaches that of a direct call. The benefits, in other words, accrue for free. Providing that path, safely and without breaking the abstraction, is what `fast_send` is for, and it is the mechanism to which we now turn.

5 Background and related work

The actor model [1, 2] specifies asynchronous message passing as the sole means of inter-actor communication, and foundational and subsequent treatments present this asynchrony as essential to isolation, fault tolerance, and location transparency [2, 15]. Several systems provide a second, synchronous-looking communication path; each differs from `fast_send` in a way that bears on receiver transparency or on multi-threaded execution.

Single-threaded synchronous calls. The E language [3] offers an immediate call (`.`) returning a value and an eventual send (`<-`) returning a promise. E vats are single-threaded, so the immediate call is a conventional method invocation with no lock, no cross-thread execution, and no possibility of deadlock; the model does not extend to actors running on multiple threads. AmbientTalk [4] similarly builds on communicating event loops in which cross-actor communication is always asynchronous. Actor4j [11] optimizes messaging between actors on the *same* thread, and reliable-actor systems such as the Kubernetes Application Runtime (KAR) [12] bypass the queue only for an actor's calls to itself, not for cross-actor calls.

Blocking that still uses the receiver's thread. Erlang's `gen_server:call` [5] blocks the caller, but the request still traverses the receiver's mailbox and is processed on the receiver's own process; the caller's thread never executes the receiver's handler. Akka's ask operator [7] returns a future and remains asynchronous internally, routing through the mailbox, and the framework explicitly discourages blocking inside actors. Orleans [9] presents grain methods in call syntax but is asynchronous underneath (`Task`-returning), with no caller-thread execution of the grain. Swift's actors [10] require `await` for cross-actor access and prohibit synchronous cross-actor calls at the compiler level. The C++ Actor Framework (CAF) [8] dispatches handlers on a scheduler's worker pool, so the caller thread does not run the target's handler.

Direct execution without message-passing semantics. The Active Object pattern [13] interposes a proxy, activation queue, and scheduler so that the client's thread never runs the servant's method on a separate thread. The Monitor Object pattern [14] does run a method on the caller's thread under a lock — sharing the caller-thread-execution mechanism with `fast_send` — but a monitor is a passive data structure invoked by direct method call: there is no mailbox, no asynchronous alternative, no dual mode, and no notion of the callee as an isolated actor that cannot observe how it was reached.

Deterministic reactor concurrency. A separate recent line of work constrains, rather than accelerates, actor-style concurrency: Lingua Franca [16] schedules reactors under a logical-time model to make concurrent execution deterministic. That objective is orthogonal to `fast_send`,

which addresses the per-message cost of co-located delivery and preserves receiver transparency; the two could be combined, since `fast_send` does not alter the logical ordering of message handling.

5.1 Positioning

Across these systems, a synchronous path either (i) is confined to a single thread or same-thread actors (E, AmbientTalk, Actor4j), (ii) blocks the caller while still executing on the receiver's thread through the mailbox (Erlang, Akka, Orleans, CAF), (iii) is prohibited outright (Swift), or (iv) abandons message-passing semantics for direct method calls on a passive object (Active Object, Monitor Object). What distinguishes `fast_send` is the combination of (a) cross-thread synchronous execution — the sender's thread runs the receiver's handler while holding the receiver's exclusive-access lock — (b) under full receiver transparency, with (c) an integrated cyclic-invocation detector that makes the cross-thread synchronous path safe against the deadlock and stack-overflow failure modes it would otherwise introduce. The remainder of the paper develops these three elements.

6 The synchronous delivery mechanism

Each actor has a message handler, a mailbox, private state, and a synchronization mechanism that guarantees at most one thread executes its handler at a time (a mutex by default; a lock-free primitive is admissible). Two delivery operations share this structure.

Asynchronous delivery (`send`). The sender heap-allocates the message, enqueues it on the receiver's mailbox under the mailbox's synchronization, and returns immediately with no value. Later, a thread assigned to the receiver dequeues the message, acquires the receiver's exclusive-access lock, and runs the handler. A reply, if any, is heap-allocated, wrapped in a response message, and enqueued on the sender's mailbox.

Synchronous delivery (`fast_send`). The sender may allocate the message on its *stack*, because the sending thread blocks until processing completes and the message's lifetime is therefore bounded by the call. The sender's thread acquires the *receiver's* exclusive-access lock (blocking if another thread holds it), pushes the receiver onto a thread-local call chain (Section 8), and invokes the receiver's handler directly on the sender's thread. The handler runs its identical code and optionally calls the reply operation; on return, the receiver is popped from the call chain, the lock is released, and `fast_send` returns the reply value (or null if none was set). A stack-allocated message is reclaimed when the sender's frame unwinds.

In both modes the receiver's exclusive-access lock is held for the duration of handler execution, so the "one message at a time" guarantee holds identically under the conservative detection policy; the permissive (actor, message-class) refinement of Section 8 admits same-actor re-entrancy and relaxes it. The distinction from a naive direct call is that the synchronous path holds the receiver's exclusive-access lock and routes the reply through the same operation as the asynchronous path; it is delivery of a message that happens to execute on the sender's thread.

Unified reply routing. A single reply operation serves both modes and selects its behavior from delivery-context information rather than from any argument the handler supplies. Under asynchronous delivery it heap-allocates the value, wraps it in a response message, and enqueues it on the sender's mailbox; under synchronous delivery it heap-allocates the value and stores it in a return-value location that the blocked `fast_send` reads and returns. The value is heap-allocated in *both*

modes deliberately: if the synchronous path could stack-allocate the reply while the asynchronous path could not, the handler would have to know the mode, defeating transparency. The reply may be invoked at most once per message. An opt-in optimized reply that stores the value without heap allocation is available for code paths known to be synchronous-only; it raises an error if the message was delivered asynchronously, and by design breaks transparency, so its use is explicit.

Chained synchronous execution. If actor A synchronously invokes B, which synchronously invokes C, and so on, the entire chain executes on A's thread, which holds the exclusive-access locks of all actors in the chain simultaneously and, when those locks are uncontended, performs no context switches. Such an N-actor synchronous chain incurs $O(1)$ context switches, against $O(N)$ for the equivalent asynchronous chain, and executes on a single core, avoiding the cross-core cache-line migration of state that the asynchronous path incurs. A contended lock in the chain blocks its thread, which reintroduces a context switch at that step.

7 Receiver transparency

Receiver transparency is the property that the receiving actor's handler is written identically for both delivery modes and has no programmatic means to determine (i) whether it was reached synchronously or asynchronously, (ii) which thread is executing it, or (iii) whether the sender is blocked awaiting a reply. It is what keeps `fast_send` within the actor abstraction rather than reducing it to a locked method call.

Three elements enforce it:

1. **Identical interface.** The handler receives a message and invokes the same reply operation in both modes; there is no mode flag, and no conditional branch in the handler depends on the delivery mode.

2. **Exclusive access in both modes.** The receiver's exclusive-access lock is held throughout handler execution whether the handler runs on the receiver's own thread (asynchronous) or on the sender's thread (synchronous), so the observable execution environment — single-writer, sequential — is the same.

3. **Mode-independent reply and allocation.** The unified reply routes by delivery context with no handler involvement, and the reply value is heap-allocated in both modes, so neither the reply's behavior nor allocation side effects reveal the mode.

Two consequences follow. First, the delivery mode becomes a *sender-side* optimization: a sender may upgrade an asynchronous send to a synchronous one with no change, recompilation, or redeployment of the receiver. Second, the mechanism is distinct from a direct procedure call, in which the callee is a passive object invoked on the caller's stack with no mailbox and no exclusive-execution guarantee; here the receiver remains an isolated actor that perceives the arrival of a message and is unaware that the sender's thread is the one running it.

8 Cyclic-invocation detection

Synchronous invocation introduces a failure mode that asynchronous delivery does not have. If synchronous calls form a cycle — $A \to B \to C \to A$ — then under a non-reentrant lock the thread waits to acquire an actor's lock it already holds, and the system deadlocks; under a reentrant lock

Algorithm 1: Synchronous send with cyclic-invocation detection

thread-local : call chain C (set of actors executing synchronously on this thread)

Input: target actor B, message m

```
1 if B ∈ C then
2     return cyclic-invocation signal (exception or error indicator) // before any lock is taken
3 acquire B's exclusive-access lock
4 C ← C ∪ {B}
5 r ← B.handler(m)                                  // runs on this (the sender's) thread
6 C ← C \ {B}
7 release B's exclusive-access lock
8 return r
```

the handler re-enters with the lock's recursion count incremented, producing unbounded nested execution that overflows the stack and exposes inconsistent intermediate state. Neither is acceptable, and the cycle cannot be discovered after the offending lock is taken — by then the thread is already stuck.

8.1 Call-chain membership test

The mechanism maintains, per thread, a *call chain*: the ordered set of actors whose handlers are currently executing on that thread via synchronous invocation. Because it is thread-local, it requires no synchronization and resets automatically between tasks. It is initialized when a thread begins processing a dequeued (asynchronous) message — the dequeued actor is its first entry — and each synchronous invocation pushes its target on entry and pops it on completion. The chain may be represented as a stack or list, a hash set for $O(1)$ membership queries, a bitmap indexed by actor identifier, or a hybrid.

Before acquiring the target's lock, a synchronous send queries whether the target is already present in the chain. If it is not, the send proceeds normally; if it is, a cyclic invocation has been detected, and the mechanism signals it — by raising a cyclic-invocation exception carrying the target identity and the full chain, or by returning an error indicator — *without* acquiring the lock or invoking the handler. Algorithm 1 states the check. Because the query precedes lock acquisition and reads only thread-local state, its overhead is negligible against the cost of a lock acquisition and a handler invocation, and the chain depth is small in practice.

8.2 Refined detection with message classes

Membership keyed on actor identity alone is conservative: it rejects legitimate callbacks in which A synchronously invokes B and B synchronously invokes a *different* handler of A. A refined mode keys the call chain on (actor, message-class) pairs, where the message-class discriminator may be a type tag, a class identifier, a handler function pointer, or any code-path discriminator. A cycle is signaled only when the same pair recurs. This admits a callback pattern such as $A/\texttt{compute} \to B/\texttt{validate} \to A/\texttt{validated}$ — three distinct pairs — while still catching genuine unbounded recursion such as $A/\texttt{foo} \to B/\texttt{bar} \to A/\texttt{foo}$, in which a pair repeats. Because permissive keying allows an actor's state to be re-entered by a nested handler, reentrancy safety in that mode is

the programmer's responsibility. The policy — conservative (actor-only) or permissive (pair-based) — is configurable system-wide, per-actor, or per-invocation.

9 A family of delivery operations

The same transparent handler interface supports five delivery operations that differ only in how the sender behaves when the receiver's lock is contended or a cycle is detected (Table 2). All synchronous variants perform the cyclic-invocation check of Section 8 before attempting the lock.

Table 2: The five delivery operations. All share one receiver-transparent handler interface; they differ in sender-side behavior under contention and cycles.

Operation	Blocks	Returns	Lock unavailable	Cycle detected
`send` (async)	no	—	(enqueues)	—
`fast_send` (blocking sync)	yes	value	waits (sleep on lock)	exception
`fast_send_spin`	yes	value	waits (busy-wait)	exception
`fast_send_x`	no	value	`ActorBusy` exception	exception
`fast_send_void`	no	—	async fallback	async fallback

- **`fast_send`** blocks on the receiver's lock by sleeping, and raises a cyclic-invocation exception on a cycle. It is the default synchronous operation.
- **`fast_send_spin`** is identical except that it busy-waits on the lock, optionally issuing a central processing unit (CPU) pause instruction between attempts, trading CPU cycles for lower wake latency by avoiding the scheduler. It suits short critical sections on latency-sensitive paths with spare cores.
- **`fast_send_x`** attempts the lock without blocking: on success it runs the handler and returns the value; on contention it raises an `ActorBusy` exception, distinct from the cyclic-invocation exception, so the caller can fall back, retry, or propagate. Its execution time is bounded.
- **`fast_send_void`** attempts the lock without blocking and returns no value: on success it runs the handler; on contention *or* on a detected cycle it enqueues the message asynchronously and returns, copying a stack-allocated message to the heap first if necessary. Because it never blocks on the target lock, the operation itself cannot be the waiting party in a lock-wait deadlock; the handler it runs on success may still block or recurse. It fits notification, forwarding, pipeline, and broadcast patterns. The ratio of synchronous completions to asynchronous fallbacks is a useful contention metric.
- An adaptive variant spins for a bounded count and then transitions to blocking acquisition, combining low latency under light contention with bounded CPU use under heavy contention.

Selection among the operations is a sender-side decision governed by whether the caller needs a return value, can tolerate an unbounded wait, and has idle processor capacity available; the receiver's handler is unaffected in every case.

10 Where latency comes from: queueing, not context switching

On a latency-critical message-handling path — of which a high-frequency trading (HFT) system is the archetype — the quantity that matters is not the mean latency but the tail: the 99th, 99.9th, and 99.99th percentiles. It is a common misconception that this tail is dominated by the cost of moving a message from one thread to another. It is not. On a modern Linux server a thread-to-thread context switch costs on the order of one to a few microseconds, and that cost is roughly fixed per hop: direct measurements put it near 1.2–1.5 μs when threads are pinned to a core, rising to a few microseconds once cache-working-set effects are included [37, 39, 38]. The tail is instead produced by *queueing*: when the code that services a queue cannot keep pace with the arrival rate, the queue grows, and the time a message spends waiting grows without bound. This is a structural property of queues, independent of any particular send primitive.

10.1 The queueing argument

Model one servicing stage as an M/M/1 queue [36]: messages arrive at rate λ and are serviced at rate μ, giving utilization $\rho = \lambda/\mu$; the queue is stable only for $\rho < 1$. The mean number of messages in the system is, by Little's law,

$$L = \frac{\rho}{1-\rho}, \tag{1}$$

and the mean sojourn time (waiting plus service) is

$$W = \frac{1/\mu}{1-\rho} = \frac{1}{\mu-\lambda}. \tag{2}$$

As $\rho \to 1$ — as the servicing code approaches saturation — $W \to \infty$. The tail is governed by the same factor: in M/M/1 the sojourn time is exponentially distributed, $\Pr[T > t] = e^{-(\mu-\lambda)t}$, so its q-quantile is

$$t_q = \frac{\ln(1/(1-q))}{\mu-\lambda} = \frac{\ln(1/(1-q))}{\mu\,(1-\rho)}. \tag{3}$$

The 99.9th percentile is therefore $t_{0.999} \approx 6.9/(\mu(1-\rho))$: *every* quantile scales as $1/(1-\rho)$, and the high percentiles are fixed large multiples of the mean. Figure 1 plots this. At $\rho = 0.5$ the 99.9th-percentile latency is about 14 service times; at $\rho = 0.9$ it is 69; at $\rho = 0.95$ it is 138. The knee is sharp, and the region past it is where HFT tails are born. Concretely, a queue whose depth hovers at 0 or 1 adds negligible latency, whereas one that backs up to hundreds or thousands of messages produces tails orders of magnitude above the mean.

The design consequence is direct. The only free parameters are λ and μ. The arrival rate λ — the market's message rate — is not ours to set. The service rate μ is: it is the reciprocal of the time the servicing code takes per message. Halving the per-message service time doubles μ, which for a fixed λ moves ρ from, say, 0.9 to 0.45 and collapses the tail. This is why the aggregate speed of the queue-servicing code — not the latency of any single send — governs the tail. A single `fast_send` that saves tens of nanoseconds is uninteresting in isolation; it is decisive because, applied across every hop of the servicing chain, it raises μ enough to hold ρ far from 1 and the queue near empty. `fast_send` attacks the tail by attacking μ.

10.2 A representative HFT pipeline

Figure 2 shows the thread architecture of a representative HFT market-data-to-order pipeline, drawn from the kaspar-hft reference implementation. CME's MDP 3.0 market data arrives as two

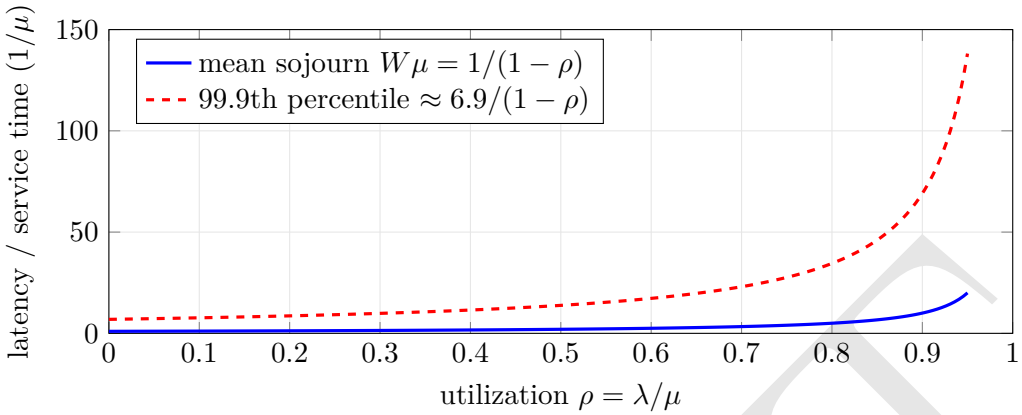

Figure 1: Latency versus utilization for an M/M/1 servicing stage, in units of the mean service time $1/\mu$. Both the mean and every tail quantile diverge as $\rho \to 1$; the tail is a fixed multiple of the mean but far larger in absolute terms. Keeping the queue short means keeping ρ well left of the knee, which means making the servicing code fast enough to raise μ.

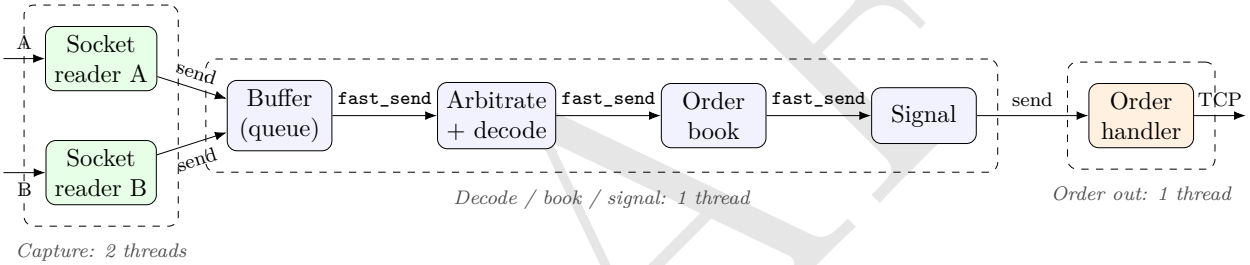

Figure 2: Thread architecture of a representative HFT pipeline. The only cross-thread hops — and hence the only context switches — on the critical path are the two asynchronous `send`s (reader→buffer and signal→order-handler); each such hop is the expensive operation, measured at $\sim$2.25 μs per round trip (Table 4), because it wakes another thread. The entire decode/book/signal servicing chain instead collapses onto a single thread via `fast_send`, each hop costing $\sim$36 ns — roughly 55–62$\times$ less (see Table 4) — so it incurs no queue and no context switch. That servicing chain is the M/M/1 stage of Section 10.1 whose per-message service time, and hence μ, is set by the aggregate cost of these `fast_send`s.

redundant multicast feeds, channel A and channel B. The system decomposes into three sections connected by exactly two cross-thread hops.

Section 1 — capture (two threads). One socket-reader actor per channel does nothing but drain its UDP socket and hand each datagram to the next stage by an *asynchronous* `send`. It must be asynchronous: a reader that made a synchronous, blocking call would stop draining its socket while the callee ran and would drop packets under a burst — defeating the reader's only purpose. In kaspar-hft the reader pre-allocates message buffers and batches whatever the socket has available before sending, and its handoff to the buffer stage is an async `send`, never a `fast_send`.

Section 2 — decode, book, and signal (one thread). The datagrams land in a buffer actor — the queue — whose service handler forwards each message down the remainder of the chain by

`fast_send`: arbitration between the A and B feeds, Simple Binary Encoding decode, order-book construction, and signal generation all run synchronously, in sequence, on this one thread, with no intervening queue and no context switch between stages. This is precisely the chained synchronous execution of Section 6: an N-stage chain with $O(1)$ rather than $O(N)$ context switches. This section is the queue-servicing code whose per-message time sets μ, and it is where `fast_send` earns its keep.

Section 3 — order out (one thread). When the signal decides to act, it hands an order to an order-management actor by an asynchronous `send`; that actor performs the TCP/iLink write. This hop is asynchronous by design: the network write takes several microseconds, and blocking the decode-and-book thread on it would stall servicing of the market-data queue — driving ρ back toward 1, the exact failure the architecture exists to avoid.

Two hops, two switches. On the critical path from wire to order there are just two cross-thread hops — reader→buffer and signal→order-out — and therefore two context switches, a fixed cost of a few microseconds in total [37]. Everything between them is collapsed onto a single thread by `fast_send`. The goal is not to remove those two switches; they are cheap and necessary, because the capture and transmit stages must not block the servicing stage. The goal is to make the servicing stage between them fast enough that its queue never grows — and that is a statement about μ, which is what `fast_send` raises. The mechanism's contribution is thus best measured not as a per-call latency but as the service rate it sustains on the servicing chain.

11 Performance analysis

The synchronous path removes, rather than optimizes, the per-message operations of the asynchronous path. Table 3 summarizes the operations eliminated for a co-located delivery.

Table 3: Per-message operations on the asynchronous path and their status on the synchronous path, for co-located actors. This is an analytical account of work removed.

Per-message operation	Asynchronous	Synchronous
Message heap allocations	1–3	0 request (stack-alloc.), 1 reply
Mailbox queue operations	2 (enqueue + dequeue)	0
Scheduler invocations	1–2	0
Context switches (request/reply)	2–4	0
Context switches, N-actor chain	$O(N)$	$O(1)$

Each eliminated item has a concrete cost. A context switch is on the order of 10^3–10^4 processor cycles and evicts cache and TLB state. Heap allocation of short-lived messages produces fragmentation and, in managed runtimes, collection pauses; stack allocation reclaims the message on frame unwind with no allocator or collector involvement. Removing the mailbox operations removes their synchronization traffic on the memory bus; quantifying that reduction requires measurement and is left to future work. The chained-execution result — $O(1)$ rather than $O(N)$ context switches across an N-actor synchronous chain — is the largest structural effect for deep call graphs.

Context: measured cost of the asynchronous path. To indicate the magnitude of the overhead the synchronous path targets, the asynchronous messaging path of the same actor runtime has been measured, in separate work [17], at 5.5 to 13.6 million round trips per second after successive

batching and memory-pooling optimizations; in the saturated throughput regime a message costs on the order of 74 ns, while in the sparse regime an individual send costs several microseconds (4–7 μs), with condition-variable wakeups costing 1–3 μs each. These are measurements of the *asynchronous* path and bound the per-message costs that synchronous delivery removes; they are not measurements of `fast_send`. We turn now to direct microbenchmarks of the synchronous path itself.

11.1 Microbenchmarks of the C++ implementation

The analytical account of Table 3 predicts that removing each per-message operation should produce a measurable step down in latency. The actor framework's C++ microbenchmark suite measures exactly these steps on a *sequential* round trip — ping → pong → reply with a single message in flight, so each sample is a clean latency rather than a saturated throughput [40]. All figures below were taken on an Apple M3 (arm64) (arm64), macOS, compiled `-O3 -march=native`, without CPU pinning, over $N = 10^6$ samples. They are *indicative rather than a specification*: absolute numbers move with hardware, allocator, and turbo state, and this is neither the x86-64 Linux target nor a pinned, quiescent machine. The *ratios* between rows — each isolating one eliminated operation — are the durable result. Because a `fast_send` is so cheap that per-sample percentiles quantize to the `steady_clock` tick ($\sim$40 ns on this platform; $\sim$10 ns on the x86-64 Linux box, so the tick is platform-specific and the bench prints the measured value rather than asserting it), its rows are reported as amortized ns (total wall time over N, one clock-read pair), which resolves costs below the tick.

We present three measurements: the cost of crossing threads (Table 4), the internal overhead of `fast_send` itself (Table 5), and the effect of the memory pool (Table 6).

Only crossing threads costs anything. Table 4 is the central result. It holds the work constant — the same ping/pong round trip with a reply — and varies only *how* the message is delivered, so each row adds back exactly one mechanism. An asynchronous `send` between two actors on *separate* threads costs about 2250 ns at the median, and that cost is almost entirely the two cross-core mailbox wakeups it performs per round trip, each a mutex acquisition plus a condition-variable signal that the receiving thread must be scheduled to service. This — not "message passing" in the abstract — is where actor latency actually comes from. Remove the thread crossing and the wakeups vanish; remove the queue as well and essentially nothing is left. The `fast_send` row is the point of the paper: at $\sim$36 ns amortized it is roughly 55–62$\times$ cheaper than the cross-thread `send` — the exact multiple depends on the metric, since `fast_send`'s per-sample $p50$ sits at the clock floor and its true cost is only resolved amortized, so this divides an amortized figure into a $p50$ and is approximate — because it is, mechanically, a locked function call. A like-for-like comparison avoids the metric mismatch entirely. Measured the same way in the same process — one clock-read pair around a tight loop of N operations, so there is no per-call clock overhead and no quantization — a bare (non-virtual) function call costs $\sim$2.9 ns and a `fast_send` of a stack-allocated message with a trivial handler costs $\sim$14 ns. That is $\sim$10 ns of overhead over a direct call: the uncontended receiver-lock acquire/release, the message field setup, the handler-cache dispatch, and the reply wrapping — on the order of one to two mispredicted virtual calls, not one. The $\sim$36 ns round-trip figure in Table 4 is larger because it additionally includes the two heap message allocations of the ping/pong pair (a stack message and the pool remove most of that; see Table 6).

Actor grouping, and why `fast_send` beats it. The middle row of Table 4 is worth dwelling on, because it is the strongest alternative and the one `fast_send` must beat. The framework lets a

Table 4: **The cost is thread-crossing, not messaging.** Same ping/pong round trip with a reply; only the delivery mechanism varies. Each row eliminates one operation relative to the row above. "Thread hop" is a cross-core mailbox wakeup (mutex + condition variable); "queue" is a mailbox push/pop. $p50/p99$ are per-sample round-trip latency except the `fast_send` row, which is amortized (see text); its speedup therefore divides an amortized figure into a $p50$ and is approximate. Ungrouped $p99$ is 6750 ns; the grouped and `fast_send` rows show no meaningful spread.

Delivery mode	Threads	Thread hop	Queue	$p50$ RT (ns)	Speedup
`send`, ungrouped	2	yes	yes	2250	1×
`send`, grouped	1	no	yes	125	18×
`fast_send` (inline)	1	no	no	∼36	∼55–62×

set of actors be placed in a `Group`: the group's actors are co-scheduled on a *single* thread behind a *single* mailbox, so an asynchronous `send` between two actors in the same group never crosses a core and never wakes another thread — it is a push and a pop on a queue already owned by the running thread. This is the conventional low-latency actor optimization — the same idea as same-thread messaging in Actor4j or a shared dispatcher in Akka (Section 3) — and it is dramatic on its own: it takes the round trip from 2250 ns to 125 ns, an 18× improvement, purely by removing the thread hop. But grouping still pays for the queue: the message is enqueued, the run loop dequeues it, and the handler runs on a later turn. `fast_send` removes that last layer. Within the same single thread, it does not enqueue at all; it takes the receiver's lock and runs the handler inline, returning the reply as a value, for ∼36 ns — about 3.5× faster than the best that grouping achieves. Grouping removes the thread hop; `fast_send` removes the thread hop *and* the queue. This second step — a synchronous, inline, receiver-transparent delivery that outperforms same-thread asynchronous messaging — is what is distinctive about the kaspar framework and is, to our knowledge, absent from the mainstream actor systems surveyed in Section 3. The intended usage follows directly: group the actors of a servicing stage onto one thread, then drive them with `fast_send`, and an N-actor pipeline collapses to a single inline call chain (the middle section of Figure 2).

The internal overhead of `fast_send` is minimal. Table 5 decomposes that ∼36 ns by varying only where the request message lives and whether a reply is sent. A bare `fast_send` that allocates nothing — a stack request, no reply — dispatches in 35.9 ns, and that is the true floor: message-id lookup, lock, and the inline handler call, with zero heap traffic. Everything above it is allocation, not dispatch. Each global `new` + `delete` of a small message adds about 15 ns; a pooled allocation adds only 2–3 ns. In other words the mechanism itself imposes essentially no overhead beyond a function call; the only variable cost a caller pays is for memory it chooses to allocate, and both stack allocation and the memory pool drive that toward zero.

The memory pool cuts the tail by two orders of magnitude. Table 6 isolates the allocator with a compile-time switch, holding the message types fixed. At the median the pool's win is modest on this platform: 124.5 ns falls to 92.8 ns (∼26%). This median win is *allocator-specific and does not travel*: on the x86-64 Linux target glibc's `tcache` is *faster* than the macOS allocator on this pattern (a global `new`/`delete` costs ∼3.6 ns there vs ∼15 ns here), so the same median win *shrinks* to ∼7.5% — the opposite of what a "slower Linux allocator" would suggest. The durable, portable benefit is not the median but the tail: in a separate run the *maximum* round trip fell from 5.5 ms to 33 μs once pooled, because the pool never takes the global allocator's occasional slow path. That is the number that matters for the queueing argument of Section 10: it is a two-order-of-magnitude

Table 5: **`fast_send` overhead is dominated by allocation, not dispatch.** Four `fast_send` configurations, amortized ns per operation, pool on. Backing out the rows: base dispatch ≈ 36 ns; each global `new`+`delete` ≈ 15 ns; each pooled allocation ≈ 2–3 ns. With the pool off, *pooled input + reply* rises to 65.6 ns, matching *heap input + reply*.

`fast_send` variant	Message allocations	Amortized (ns/op)
stack input, no reply	none	35.9
pooled input + reply	2 pooled	41.3
stack input + reply	1 global (the reply)	51.1
heap input + reply	2 global	65.8

cut to the exact $p99.9+$ tail that a slow servicing stage feeds into a growing queue.

Table 6: **Global allocator vs `MemoryPool`**, grouped `send`, amortized ns per round trip (two allocations per round trip). The compile-time switch `DISABLE_MEMORY_POOL` is the only difference between columns; with it off, pooled types collapse to the plain `new`/`delete` cost, confirming the switch is all that changes. Tail (separate run): maximum round trip 5.5 ms → 33 μs.

Message source	Pool on (ns/RT)	Pool off (ns/RT)
plain `new`/`delete`	124.5	128.1
`MemoryPool`	**92.8**	128.2

Guidance, and what is not shown. The measurements yield a mechanical rule for hot-path callers: group the actors of a stage onto one thread; deliver with `fast_send`; keep the request message on the stack when possible (it need never be heap-lived, because `fast_send` blocks until the handler returns, Section 6); and for any message that must outlive the call, draw it from a `MemoryPool`, turning a global allocation (∼ 15 ns on macOS, ∼ 3.6 ns on glibc) into a ∼ 2–3 ns pooled one and, above all, removing the allocator tail — the one part of the pool's benefit that is platform-independent. What these numbers do *not* establish is equally important: they are sequential latencies with one message in flight, not saturated throughput; they do not vary contention or chain depth; and they were taken primarily on an Apple M3. A pinned cross-check on the x86-64 Linux target (AMD EPYC 9374F, `g++` 15) reproduces the *ratios* — grouping still collapses the round trip, `fast_send` still removes what remains, and its overhead over a bare call still measures single-digit nanoseconds (+9 ns there, about *one* unpredictable virtual call) — but not the allocator or clock-resolution *absolutes* above. That multi-axis, saturated evaluation — the subject of Section 12 — remains future work. What is established is that `fast_send` costs tens of nanoseconds, that its overhead is a function call plus whatever memory the caller chooses to allocate, that it beats even same-thread asynchronous grouping by roughly 3.5×, and that pooling removes the latency tail.

12 Discussion and limitations

The scope is co-located actors. `fast_send` applies only when sender and receiver share an address space; a synchronous cross-machine call would reintroduce the network round trip it exists to avoid, and the actor model's location transparency means the sender can fall back to asynchronous delivery when the receiver is remote. Receiver transparency is what makes that fallback free: the same handler serves the remote asynchronous case and the local synchronous case.

The cost of transparency. Heap-allocating the reply in both modes is a deliberate cost paid to keep the handler unable to distinguish them. The optimized synchronous-only reply removes that allocation but forfeits transparency and must therefore be confined to code paths statically known to be synchronous; mixing it with asynchronous delivery is an error the mechanism reports rather than tolerates.

Blocking runs counter to conventional guidance. Established actor frameworks discourage blocking inside actors precisely because it can starve thread pools and cause deadlock [7, 6, 10]. `fast_send` blocks the sender by construction, and its safety rests on the cyclic-invocation detector and on the availability of the non-blocking and best-effort variants for paths that cannot tolerate an unbounded wait. Where a caller must never block, `fast_send_x` and `fast_send_void` provide bounded-time and never-blocking alternatives behind the same handler interface. Even so, the cyclic-invocation detector addresses only *cyclic* waits; avoiding thread-pool starvation for acyclic blocking patterns — a chain that blocks on an actor whose handler is itself waiting on a scarce resource — remains a burden on the developer that asynchronous delivery does not impose.

Contention on a hot actor. A synchronous send holds the receiver's exclusive-access lock for the duration of handler execution, so concurrent synchronous sends to the same heavily contended actor serialize on that lock, and the blocking variants stall their caller threads while they wait. Under heavy contention this can perform worse than asynchronous delivery, in which senders enqueue and continue and the messages buffer without stalling the callers. The non-blocking (`fast_send_x`) and best-effort (`fast_send_void`) variants bound or remove the stall — the latter by falling back to the mailbox — so the ratio of synchronous completions to asynchronous fallbacks, a metric the mechanism already exposes, doubles as a signal that a target has become hot enough that the asynchronous path is preferable.

Permissive detection shifts a burden to the programmer. The (actor, message-class) refinement admits callbacks by allowing an actor's state to be re-entered by a nested handler, which reintroduces the reentrancy hazards that strict per-actor sequential processing removes. A nested handler can observe and mutate the actor's state mid-update, which relaxes the actor model's guarantee of single-threaded sequential state mutation and admits the invariant violations that guarantee is designed to prevent. The conservative policy avoids this at the cost of rejecting some legitimate callback structures; the choice between them is exposed as configuration rather than fixed.

Further evaluation. The microbenchmarks of Section 11.1 establish the per-operation latency reductions and the two-order-of-magnitude tail benefit of the memory pool directly. The natural next step measures the mechanism under load: synchronous versus asynchronous *throughput* across contention levels, chain depths, and message sizes on the x86-64 Linux target, together with the distribution of synchronous completions versus asynchronous fallbacks for `fast_send_void`. That saturated, multi-axis characterization is the subject of ongoing work.

13 Conclusion

Asynchronous message passing is intrinsic to the actor model's guarantees and indispensable across machines, but for co-located actors it imposes heap allocation, queue operations, scheduling, and context switches that isolation does not require. `fast_send` provides a synchronous alternative in

which the sender's thread runs the receiver's handler directly under the receiver's exclusive-access lock and takes the reply as a value, while the handler remains unable to determine the delivery mode, the executing thread, or the sender's blocked state — receiver transparency. A thread-local call-chain membership test, checked before lock acquisition, makes the cross-thread synchronous path safe against the deadlock and stack-overflow modes that cyclic synchronous invocation would otherwise cause, and a refinement keyed on (actor, message-class) pairs admits legitimate callbacks while still catching unbounded recursion. Five delivery operations expose the latency/robustness spectrum behind one transparent handler interface. The performance case is made analytically, as the elimination of per-message work, and corroborated by microbenchmarks in which the synchronous path completes a round trip in tens of nanoseconds and a memory pool cuts the latency tail by two orders of magnitude; a saturated-throughput evaluation under contention remains to be done.

Disclosure

The `fast_send` mechanism and its cyclic-invocation detection are the subject of a pending patent application. This paper is a technical description and does not constitute legal advice.

References

[1] C. Hewitt, P. Bishop, and R. Steiger. *A Universal Modular ACTOR Formalism for Artificial Intelligence.* Proc. 3rd International Joint Conference on Artificial Intelligence (IJCAI), 1973.

[2] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems.* MIT Press, 1986.

[3] M. S. Miller, E. D. Tribble, and J. Shapiro. *Concurrency Among Strangers: Programming in E as Plan Coordination.* Symposium on Trustworthy Global Computing, 2005.

[4] T. Van Cutsem et al. *AmbientTalk: Object-Oriented Event-Driven Programming in Mobile Ad Hoc Networks.* XXVI Int. Conf. of the Chilean Computer Science Society, 2007.

[5] J. Armstrong, R. Virding, C. Wikström, and M. Williams. *Concurrent Programming in Erlang.* Prentice Hall, 2nd ed., 1996.

[6] J. Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors.* PhD thesis, Royal Institute of Technology (KTH), Stockholm, 2003.

[7] Lightbend, Inc. *Akka Documentation.* https://akka.io/docs/.

[8] D. Charousset, T. C. Schmidt, R. Hiesgen, and M. Wählisch. *CAF — The C++ Actor Framework for Scalable and Resource-Efficient Applications.* Proc. 4th Int. Workshop on Programming based on Actors, Agents, and Decentralized Control (AGERE!), 2014.

[9] S. Bykov et al. *Orleans: Cloud Computing for Everyone.* Proc. 2nd ACM Symposium on Cloud Computing (SoCC), 2011.

[10] Apple, Inc. *Swift Evolution Proposal SE-0306: Actors.* 2021.

[11] D. Bauer. *Actor4j: A Software Framework for the Actor Model Focusing on the Optimization of Message Passing.* 2018.

[12] O. Tardieu et al. *Reliable Actors with Retry Orchestration.* IBM Research, 2021.

[13] R. G. Lavender and D. C. Schmidt. *Active Object: An Object Behavioral Pattern for Concurrent Programming.* Pattern Languages of Program Design 2, Addison-Wesley, 1996.

[14] D. C. Schmidt, M. Stal, H. Rohnert, and F. Buschmann. *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects.* Wiley, 2000.

[15] H. Sutter and J. Larus. *Software and the Concurrency Revolution.* ACM Queue, 3(7):54–62, 2005.

[16] M. Lohstroh, C. Menard, S. Bateni, and E. A. Lee. *High-Performance Deterministic Concurrency using Lingua Franca.* arXiv:2301.02444, 2023.

[17] V. Maciejewski. *How Message Batching More Than Doubles Actor-Model Throughput.* https://vincentmayeski.substack.com/p/how-message-batching-more-than-doubles, 2026.

[18] I. G. Greif. *Semantics of Communicating Parallel Processes.* PhD thesis, MIT, 1975. Project MAC Technical Report MAC-TR-154.

[19] C. Hewitt and H. Baker. *Laws for Communicating Parallel Processes.* In *Information Processing 77* (Proc. IFIP Congress 77), North-Holland, 1977, pp. 987–992.

[20] W. D. Clinger. *Foundations of Actor Semantics.* PhD thesis, MIT, 1981. MIT AI Lab Technical Report AI-TR-633.

[21] J. Armstrong. *A History of Erlang.* Proc. 3rd ACM SIGPLAN Conference on the History of Programming Languages (HOPL III), 2007. https://doi.org/10.1145/1238844.1238850.

[22] C. Hewitt. *Actor Model of Computation: Scalable Robust Information Systems.* arXiv:1008.1459, 2010 (rev. 2015). https://arxiv.org/abs/1008.1459.

[23] E. Stenman. *The Erlang Runtime System (The BEAM Book), chapter "Scheduling".* https://github.com/happi/theBeamBook, accessed 2026.

[24] Lightbend (Akka Team). *50 Million Messages per Second — on a Single Machine.* https://letitcrash.com/post/20397701710/50-million-messages-per-second-on-a-single, 2012.

[25] S. Bykov, A. Geller, G. Kliot, J. R. Larus, R. Pandya, and J. Thelin. *Orleans: Distributed Virtual Actors for Programmability and Scalability.* Microsoft Research Technical Report MSR-TR-2014-41, 2014.

[26] S. Clebsch, J. Franco, S. Drossopoulou, A. M. Yang, T. Wrigstad, and J. Vitek. *Orca: GC and Type System Co-design for Actor Languages.* Proc. ACM Program. Lang. 1(OOPSLA), Article 72, 2017.

[27] C. Camilleri, J. G. Vella, and V. Nezval. *Actor Model Frameworks: An Empirical Performance Analysis.* In *Advances in Information and Communication* (FICC 2023), Lecture Notes in Networks and Systems, vol. 652, Springer, 2023. https://doi.org/10.1007/978-3-031-31153-6_37.

[28] S. M. Imam and V. Sarkar. *Savina — An Actor Benchmark Suite: Enabling Empirical Evaluation of Actor Libraries.* Proc. 4th Int. Workshop on Programming based on Actors, Agents, and Decentralized Control (AGERE!), 2014. https://doi.org/10.1145/2687357.2687368.

[29] S. Blessing, K. Fernandez-Reyes, A. M. Yang, S. Drossopoulou, and T. Wrigstad. *Run, Actor, Run: Towards Cross-Actor Language Benchmarking.* Proc. 9th Int. Workshop on Programming based on Actors, Agents, and Decentralized Control (AGERE!), 2019.

[30] D. Charousset, T. C. Schmidt, R. Hiesgen, and M. Wählisch. *Native Actors — A Scalable Software Platform for Distributed, Heterogeneous Environments.* arXiv:1301.0748, 2013.

[31] Actix Project. *Actix: Actor Framework for Rust — Documentation.* https://docs.rs/actix/, accessed 2026.

[32] S. Tasharofi, P. Dinges, and R. E. Johnson. *Why Do Scala Developers Mix the Actor Model with Other Concurrency Models?* Proc. 27th European Conference on Object-Oriented Programming (ECOOP), LNCS 7920, Springer, 2013, pp. 302–326.

[33] M. Bagherzadeh, N. Fireman, A. Shawesh, and R. Khatchadourian. *Actor Concurrency Bugs: A Comprehensive Study on Symptoms, Root Causes, API Usages, and Differences.* Proc. ACM Program. Lang. 4(OOPSLA), Article 214, 2020. https://doi.org/10.1145/3428282.

[34] C. Torres Lopez, S. Marr, H. Mössenböck, and E. Gonzalez Boix. *A Study of Concurrency Bugs and Advanced Development Support for Actor-based Programs.* In *Programming with Actors*, LNCS 10789, Springer, 2018, pp. 155–185. Also arXiv:1706.07372.

[35] J. Huang, T. Mahmud, C. Păsăreanu, and G. Yang. *CONCUR: Benchmarking LLMs for Concurrent Code Generation.* arXiv:2603.03683, 2026.

[36] L. Kleinrock. *Queueing Systems, Volume 1: Theory.* Wiley-Interscience, 1975.

[37] C. Li, C. Ding, and K. Shen. *Quantifying the Cost of Context Switch.* Proc. Workshop on Experimental Computer Science (ExpCS), ACM, 2007.

[38] L. McVoy and C. Staelin. *lmbench: Portable Tools for Performance Analysis.* Proc. USENIX Annual Technical Conference, 1996.

[39] E. Bendersky. *Measuring Context Switching and Memory Overheads for Linux Threads.* https://eli.thegreenplace.net/2018/measuring-context-switching-and-memory-overheads-for-linux-threads/, 2018.

[40] V. Maciejewski. *kaspar-hft actor framework: C++ message-passing microbenchmarks (`actors/cpp/perf`).* https://github.com/vincent212/kaspar-hft, 2026.